1 主存储（核心数据）

nodes
HashMap<NodeId, ResponsibilityNode>
存储所有责任节点

edges
HashMap<EdgeId, ResponsibilityEdge>
存储所有责任边

2 六大索引（加速查询与遍历）

1 **agent_nodes**
AgentId → Vec<NodeId>
给定 Agent 查询关联节点
由 add_node 插入时更新

2 **agent_edges**
AgentId → Vec<EdgeId>
由边两端 Agent 更新
source=target 时避免重复

3 **outgoing_edges**
NodeId → Vec<EdgeId>
给定源节点查询出边
用于 bfs_from 正向遍历

4 **incoming_edges**
NodeId → Vec<EdgeId>
给定目标节点查询入边
用于 walk_backward 追溯

5 **nodes_by_type**
NodeType → Vec<NodeId>
按节点类型分类查询

6 **edges_by_type**
EdgeType → Vec<EdgeId>
按边类型分类查询

3 FIFO 淘汰机制（容量控制）

node_order（Vec<NodeId>）
节点插入顺序队列（FIFO）

n_1 n_2 … n_k
头部（最早） 尾部（最新）

edge_order（Vec<EdgeId>）
边插入顺序队列（FIFO）

e_1 e_2 … e_m
头部（最早） 尾部（最新）

- add_node：追加到 node_order，随后调用 **enforce_node_limit**
- add_edge：追加到 edge_order，随后调用 **enforce_edge_limit**
- 超过 hard_max_nodes / hard_max_edges 时，
  从队列头部逐个移除；remove_node 同步清理索引与关联边

4 七种节点类型与结构（ResponsibilityNode）

Agent
node_id
agent_id
evidence_refs
timestamp_ns

Resource
resource: NormalizedRes.
资源规范化描述
其余字段同结构体

PolicyBoundary
策略边界/约束域
边界跨越责任分析
其余字段同结构体

Process
进程/执行上下文
追踪进程生命周期
其余字段同结构体

ToolCall
工具调用节点
记录 Agent 的工具调用
其余字段同结构体

Credential
凭证/身份节点
身份凭证/认证信息
其余字段同结构体

Context
上下文/环境节点
环境信息/会话上下文
其余字段同结构体

5 十二种边类型与结构（ResponsibilityEdge）

- **IntentCausesAction** 意图→行动（因果/触发）
- **AgentDelegatesTo** 父→子Agent（委托关系）
- **ContextFlowsFrom** 源→目标节点（上下文流动）
- **TaintPropagatesTo** 资源→Agent（污点传播）
- **PolicyBoundaryCrossing** 跨策略边界
- **CompetingExplanation** 竞争性解释
- **Delegation** 委托关系
- **AuthorityTransfer** 权限转移
- **CapabilityUse** 能力使用
- **ContextInheritance** 上下文继承
- **TaintPropagation** 污点传播
- **ViolationProvenance** 违规溯源

edge_id: EdgeId　edge_type: RespEdge　confidence: f32
source: NodeId　target: NodeId　phase: PhaseIndicator
evidence_refs: Vec<EvidenceId>　signal: Option<DetectedSignal>

图结构示例

